PREDICTING THE OUTCOME OF EPL FOOTBALL MATCHES

Group Name: `Group D`
Department of Computer Science
University College London
London, WC1E 6BT

March 2, 2021

1 Introduction

We aim to predict the results of EPL matches taking place on the 16th of January 2021. We accomplished this by using various Machine learning techniques on both data provided to us and data that we gathered from the Internet.

2 Data Transformation & Exploration

2.1 Provided Data

The first thing we did to the data given to us was to check the frequency of each outcome in the training dataset. This is because we suspected the dataset was imbalanced [1], as we expected draw to have a much lower frequency that the other two outcomes. However, as Figure 1 illustrates, we were surprised to see that home wins dominated away wins by quite a significant margin. This was important for us, as it informed us that when we were evaluating our models, we should be wary of just looking at accuracy. This is further expanded upon in the Section 5.

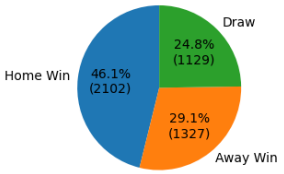


Figure 1: Distribution of results

Additionally, from our dataset, we discovered that:

- The shape of the dataset is 4560×73.
- Many columns are completely empty.
- The dates are inconsistent, as they have two different formats: "dd-mm-yy" and "dd-mm-yyyy".

We concluded that some data cleaning needs to be done.

2.1.1 Data Cleaning

- We removed the empty columns, resulting in a new shape of the dataset - 4560×22.
- We unified the date format to "yyyy-mm-dd".
- We used df.isnull().sum() to make sure that there is no missing value in order dataset.

We decided to drop referee column as it is a feature that is very difficult to encode: Not only are there more than 42 referees in the dataset, but it would be hard to encode new referees for later predictions. Also, upon further research, we found that referee bias generally favored the home team, [9] so we reasoned that this bias would be captured by our model anyway, giving us more confidence that dropping the referee column would not affect the accuracy too badly.

2.1.2 Data Preprocessing

We observed that there are over 20 features for each match. However, apart from the date and the participating teams, we don't have any of the other features when we want to predict the upcoming matches, and nor can we obtain these

numbers before the match starts. We then decided to try and map existing features, such as averaging the total goals scored in previous games, into new features such as attack strength. This is useful, as we could then use this data in our actual predictions (recency features). We go into depth about the means of obtaining these features in the Methodology section (Section 3)

We also one-hot encoded the result of each match by adding three columns of "HomeWin", "AwayWin", and "Draw" to the dataset. We set the values of these columns to 1 if a match ends up in this result, and 0 otherwise. We also normalized all other features. This encoding is done to remove the bias for our Neural Network models.

We now have features as shown in Table 1 and Table 2.

Table 1: Features of the Match between Arsenal and West Brom on 16 Aug, 2008

Dist	ATMV	HTMV	HA0	HD0	HOS0	HH0	AA0	AD0	AOS0	AH0	...	HELO	AELO	HWR
169.50	57.65	225.77	1.248	1.486	-0.380	-0.570	1.477	1.232	-0.381	0.572	...	1500	1500	0.5

ATMV: AwayTeamMarketValue, HTMV: HomeTeamMarketValue, HA: HomeTeam attacking strength, HD: HomeTeam defensive strength, HOS: HomeTeam strength of opposition, HH: HomeTeam home advantage AA: AwayTeam attacking strength, AD: AwayTeam defensive strength, AOS: AwayTeam opposition strength, AH: AwayTeam home advantage, 0-4 represents each of the previous 5 matches.

Table 2: Labels of the Match between Arsenal and West Brom on 16 Aug, 2008

HomeTeamWin	AwayTeamWin	Draw
1	0	0

We had a quick analysis of the features by using the Pearson Correlation Heatmap. Figure 2 shows the 10 most related features that affects the match result, including AELO, AwayTeamMarketValue, away team attacking strength in the previous five matches, and home team defensive strength in some of the previous five matches. It seems that the aggressiveness of the away team seems to be a very important factor that will affect the match result.

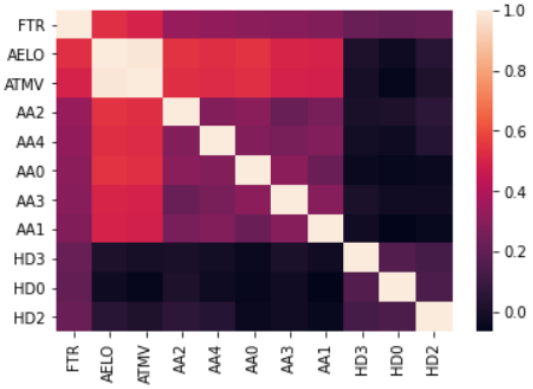


Figure 2: Pearson Correlation Heatmap

2.2 Extra Data

2.2.1 Extra Numerical Data

We concluded that some other factors might impact the performance of the teams. Following is a list of the extra features we built. We go into more depth about how we gathered and processed these novel features in section 3.2.

- Market value
  In most cases, market value is a direct reflection of how strong a team is. We used web crawler to crawl the data from the Internet.
  Source: `https://www.transfermarkt.co.uk/premier-league/marktwerte/wettbewerb/GB1`

- Distance between home and away teams
  Away teams have to travel to a different city, which could be tiring for the players, affecting their performance. We used a Python package called pgeocode to calculate these distances. Description of the package can be found at `https://pypi.org/project/pgeocode/`
- Elo Ratings
  We calculated the Elo ratings of each team in the training set. Elo ratings reflect the strength of the teams with respect to the results of all previous matches. It works by alerting two parties ratings after they play against each other in a match. Section 3 describes this algorithm in more detail.
- Recency Features
  We referred to [4] and used the results of five previous matches of both competitors as features, attempting to capture the abilities of scoring goals and defending against goals of two teams.
- Psychology of players
  We tried to factor in the mindset of players on each team into our models. We used twitter as a source of data to do this.

3 Methodology

3.1 General Overview

The problem at hand is to predict which of the three classes Home win (H), Away win (A), Draw (D) a match falls into. Hence, we have a classification problem as the output is finite and discrete. We split our dataset into a training dataset and a testing dataset. The ratio of test to train data that we have chosen is 20:80. We then considered the different types of classification models we could use, which were:

1. Logistic Regression
2. Stochastic Gradient Descent
3. Decision Tree Classification
4. Random Forest Classification
5. Linear Discriminant Analysis
6. Quadratic Discriminant Analysis
7. k Nearest Neighbors
8. Support Vector Machine
9. Deep Neural Network

We chose these models as they could be used for multi-class classification. After training and predicting the results, we evaluated the models using the performance scores. These performance scores are the F1-Score, roc-auc score, accuracy and cross-validation score. We didn't rely solely on accuracy as this could be insufficient to reflect the suitability of a model [8].

Hyper-parameters of our models were then tuned using GridSearchCV, and we observed the performance scores once again to see if the optimisations done were valuable.

After optimisations, we used Principal Component Analysis (PCA) to reduce the dimensionality of our design matrix [10]. The process removes correlated features (features influenced by some common mechanism and tend to vary together) and can make the algorithm less computationally intense. Although PCA enabled us to improve our algorithm and reduce computational complexity, there were drawbacks to it as well. Our original features were turned into principal components, which were linear combinations of our original features, thus making it less readable and harder to interpret. [11].

We then trained the models once again using the new features and compared them to the old models. Finally, we analysed the scores and chose the highest performing models to form a committee [15] to make our final predictions.

3.2 Additional features

We crafted our new features to try and target any factors that influence a football game which was not already captured by the data. We then came up with ways to map this information into features so that our models could analyse them. These features are mentioned below.

3.2.1 Psychology of players

When we were looking at the features we were given, we noticed that none of them were able to characterize an important factor in sport performance – the team's psychology. This is undoubtedly a crucial feature in sport performance. [5] We then decided to try and engineer our own feature which measures the teams state of mind before they play a football

game. We decided to use twitter as a medium to do this. This is because most athletes have a twitter account, allowing us to analyze what they say before games using natural language processing (nlp) and sentiment analysis. Moreover, twitter's API allows for the scraping of tweets, which would help facilitate gathering this information.

We then had to think about how we could incorporate the sentiment information from the tweets into the data set. Each entry in the dataset is basically determined by two teams and a date, so we had to map individual sentiment scores from the players into these records. To do this, we first built a web scraper to obtain a list of the player roster for every team every season [we used the following website to do this - https://www.premierleague.com/clubs/]. We then wrote a program that found the twitter handle for each of the players, and scraped every tweet the players posted in the designated time frame. We made sure to translate these tweets, as many of the players posted in different languages.

Once we had the text from the tweet, we did some sentiment analysis on it. We chose to use a lexicon and rule-based sentiment analysis tool called Vader [6], since its dataset was built specifically for social media text. This produced a compound, continuous, scaled sentiment score that illustrated the degree of positivity or negativity in the text. We then stored the score along with the player name and date in a text file. We did not store the actual text itself, as it would have resulted in an enormous file. An example of the sentiment scores are demonstrated in Figure 3:

```
(base) samis-MacBook-Pro:twitter scraper samialalawi$ python3 scaper.py
          player                                                tweet  sentiment       date
0  Wilfred Ndidi  RT @LCFC: A happy anniversary https://t.co/f...     0.5719 2021-01-03
1  Wilfred Ndidi  Great to start the year with a win https://...     0.8360 2021-01-03
2  Wilfred Ndidi            HAPPIER 2021 https://t.co/qRLQrLQcp4     0.6289 2020-12-31
3  Wilfred Ndidi  Amazing feeling to get 3 points and a clean sh...     0.7906 2020-12-20
4  Wilfred Ndidi  RT @Steve_Crossman: Nice chat with @LCFC + @th...     0.4215 2020-12-15
5  Wilfred Ndidi  Another 3 points and a clean sheet to start a ...     0.4019 2020-12-13
6  Wilfred Ndidi    Does it come with ps5? https://t.co/RUMYudWhGy     0.0000 2020-12-09
7  Wilfred Ndidi  All smiles for the win https://t.co/EoCefPfTbs     0.7845 2020-12-06
8  Wilfred Ndidi                          https://t.co/44mliecLOZ     0.0000 2020-12-02
9  Wilfred Ndidi                          https://t.co/njmHEnl1te     0.0000 2020-12-02
```

Figure 3: Example of NLP

Finally, for each row in the original data set, we looked up the home and away team players at that time, and looked at the index of their tweet's sentiment scores three weeks before the match start date (excluding the day of the match, as the players could have tweeted after the match about its results, which would have led to bias results). We then summed the sentiment for each team, and included this as an additional feature in the dataset.

We have included the code for this entire process at the end of the notebook, to show exactly how we carried out each step in this process. To run the code, you need a twitter developer account, and need to plug in the credentials to get the twitter API running. We initially completed the stages in batches, but we estimate that running it all at once would take about 70 hours on an ordinary computer.

We are aware that there are some drawbacks with using this feature. Primarily, not all of the players have a twitter account, and this could lead to bias results in the feature we have implemented. Also, on closer inspection of the dataset, there are many empty values for the earlier years (08,09 etc), as players were less active on twitter in this time frame, which could affect the accuracy of our models.

3.2.2 Twitter extra use case

As an additional feature, we also wanted to try and use twitter to scrape some fan tweets about the matches in order to predict their outcomes. We were inspired to do this as a result of a paper we read during the course last year [7] which introduced swarm intelligence. We saw a parallel here, where people on twitter would act as the agents whose task is to produce a prediction of the football outcome.

However, we decided against doing this for two reasons. Firstly, unlike the previous twitter scraping, we would need to search for a topic rather than specific user accounts. Unfortunately, twitter only allows users to pull tweets of this kind up to 30 days ago, which would hinder our ability to get the data required for previous matches. Additionally, we reasoned that football match prediction would be less susceptible to the wisdom of the crowds due to people's personal bias. For example, clubs with bigger fan bases would get more support and positivity online, even if realistically they are unlikely to win. This would have introduced bias into our feature, so for these two key reasons, we decided not to include fan support as a feature.

3.2.3 Recency Features

A paper we referred to was [4], in which the author used recency features of two competitors to predict match outcomes. Recency features are obtained from past n matches of the two teams and include:

- Attack Strength ({H,A}A{0,1,2,3,4}): Number of goals scored by the team in the past n matches

- Defense Strength ({H,A}D{0,1,2,3,4}): Number of goals scored by its opponents
- Opposition Strength ({H,A}OS{0,1,2,3,4}): Average goal difference of its opposition in past n matches
- Home Advantage ({H,A}H{0,1,2,3,4}): 1 if the team was the home team; 0 otherwise

In this project we adopted 5 for n. For unavailable data, (e.g. a team might not have past n matches in EPL) we assigned average value of that column to it.

For every match, we used FTHG and FTAG to calculate the four strength-related features for both teams, and these features will be reflected in the following few matches that either of the teams participates in.

3.2.4 ELO

Elo rating is an algorithm that can calculate the relative skill level of the participants. It was originally used in chess but later on used in other sports as well, such as esports, football and basketball [2].

We implemented this algorithm by referring to [3]. The algorithm can generate the elo rating of home team and away team of a match as well as the win rate of the home team, which could be used as some useful features for training the model.

Elo ratings are calculated as follows:

Given Elo ratings of two teams, r_1 and r_2, , the win rate E_1 and E_2 of two teams are given by:

$$\text{let } R_1 = 10^{r_1/400},\ R_2 = 10^{r_2/400}$$
$$E_1 = \frac{R_1}{R_1 + R_2}, E_2 = \frac{R_2}{R_1 + R_2}$$

Given the result of one match, the new Elo ratings of two competitor, r_1' and r_2', are calculated as follow:

$$\text{let } S_1 = \begin{cases} 1, & \text{if team 1 wins} \\ 0.5, & \text{if the match draws} \\ 0, & \text{otherwise} \end{cases},\ S_2 = 1 - S_1$$
$$r_1' = r_1 + K * (S_1 - E_1), r_2' = r_2 + K * (S_2 - E_2)$$

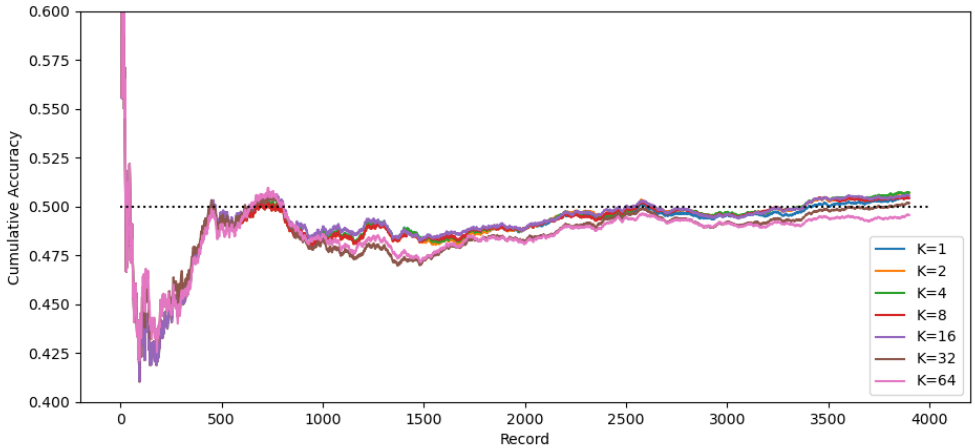


Figure 4: Distribution of results

Where K is a constant and its optimal value is found to be $K = 1$ in terms of cumulative accuracy in predicting match outcomes (Figure 4)

Table 3: Elo Rating of the first three matches Arsenal participated in

Date	HomeTeam	AwayTeam	FTR	HELO	AELO	HWR
2008/8/16	Arsenal	West Brom	H	1500	1500	0.5
2008/8/23	Fulham	Arsenal	H	1500.5	1500.5	0.5
2008/8/30	Arsenal	Newcastle	H	1500.001	1500.501	0.499281

FTR: Full time result, HELO: HomeTeam ELO, AELO: AwayTeam ELO, HWR: HomeTeam Win Rate

Table 3 demonstrates how ELO rankings work, using Arsenal as an example. We started by giving Arsenel an initial rating of 1500. They won the first match against West Brom, so by the time they participated in the second match, their rating had risen to 1500.5. After they lost to Fulham in the second match, their rating dropped to 1500.001, as reflected in the third match.

4 Model Training & Validation

We ran the models with the default starting hyperparameters. The results are shown in Table 4. Since the hyperparamters were not tuned, and the models were not regularised, we reasoned that there was some optimisations we could do to improve our accuracy even further.

Table 4: Model Scores Pre-optimisation

Model	Accuracy	F1 score	cv_score	roc_auc score	Mean
Logistic Regression	0.527	0.469	0.526	0.647	**0.542**
SGDClassifier	0.338	0.341	0.376	0.500	**0.389**
Decision Tree Classifier	0.409	0.412	0.428	0.542	**0.448**
Random Forest Classifier	0.535	0.445	0.523	0.664	**0.542**
Linear Discriminant Analysis	0.533	0.481	0.526	0.655	**0.549**
Quadratic Discriminant Analysis	0.453	0.453	0.464	0.614	**0.496**
KNN	0.415	0.387	0.455	0.554	**0.453**
Support Vector Machine	0.533	0.470	0.519	0.631	**0.538**
Deep Neural Network	0.522	0.423	N/A	N/A	**0.473**

4.1 Hyperparameter Optimization

4.1.1 Grid Search vs. Random Search

There are two popular approaches when performing hyperparameter optimization: Grid Search and Random Search, both having the same input - a dictionary of hyperparameters. For example, when we do grid search on Decision Tree Classifier, an input may look like this:

$\{'criterion' : ['gini','entropy'],'splitter' : ["best","random"],'max_features' : [None,'sqrt','log2']\}$

In Grid Search, for each combinations of hyperparameters, we train a model and evaluate it using our testing dataset and select the best model. On the other hand, Random Search randomly selects combinations of hyperparameters. Random Search is more time efficient than Grid Search, however, it is not guaranteed to find the optimal combination of hyper parameters. We decided to adopt Grid Search in our project as we are interested in maximising our models performance.

4.1.2 Deep Neural Network

The performance of the DNN model is closely related to hyperparameters, as they determine the structure of the DNN and how the model will be trained. The hyperparameters are:

- number of hidden layers
- number of neurons on each hidden layer
- dropout rate
- learning rate
- activation function on each hidden layer
- number of epochs
- batch size

The number of hidden layers we tried were 2,3,4. We chose this range because too few layers would result in under-fitting, with no hidden layers resulting in linear separable functions, and one hidden layer resulting in the model representing functions that contains a continuous mapping from one finite space to another. [12] On the other hand, too many layers can result in over-fitting and an increased training time.

The number of neurons we tried were 128, 256, 512. We didn't have any idea about how many neurons worked best, so we experimented by creating a function which takes the number of layers as an input and return all possible combinations of neurons for each layer. For example, if the input is 2, the function will return [(128,128), (128, 256), (128, 512), (256, 128), (256, 256), (256, 512), (512, 128), (512, 256), (512, 512)]

The learning rates we tried were 0.0005 and 0.00005. The trade off here is that a large learning rate may prevent the optimizer from converging, whereas, a small learning rate takes longer to converge.

We decided not to do grid search on activation functions. This is because we know the characteristics of activation functions we need:

- Non-linear
- Computationally efficient

Thus, Rectified Linear Unit (ReLU) seems to be suitable for our model. However, "one downside of ReLU is that if inputs is negative, the backpropagation cannot be performed" [13].

Thus, we chose Leaky ReLU as our activation functions on hidden layers.

The number of epochs (the number times that the learning algorithm will work through the entire training dataset) is also an important hyperparameter. If it is too large, our model may be over-fitting the training data. If it is too small, our model may be under-fitting the training data.

However, we decided not to do grid search on it, as the optimal number of epochs can be determined during the process of training using the Callbacks API provided by Keras. More specifically, we defined an EarlyStopping function which monitors the loss on the validation dataset during the process of training, so once there is an increase in our loss, the training will be stopped immediately.

4.1.3 Other models

Thanks to Sklearn, we can do grid search by using its build-in function: GridSearchCV. The processes of doing grid search on models 1-8 was similar to our method with DNN.

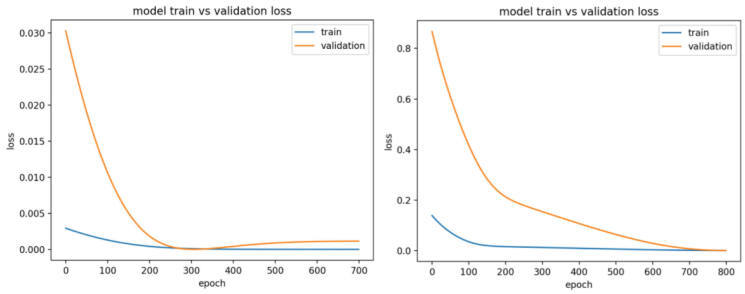
Firstly, we defined a dict object which specified the hyperparameters we want to search. Then, we passed the model object as well as our dict into GridSearchCV. The functions automatically trained each model, scored it and return the best model.

4.2 Handling Over-fitting In Deep Neural Network

Over-fitting occurs when our DNN models performs perfectly well on training data, but it does not perform well on testing data, which means that the model simply memorizes the training data and fails to develop generalisation ability. To determine whether or not a DNN model is over-fitting, we can compare the performance of the model on the testing set to the performance of the model on the training set. Figure 5a A is an example of a training history of an over-fitting model, whereas Figure 5a B is an example of a training history of a model which is not over-fitting. We adopted multiple techniques to handle over-fitting.

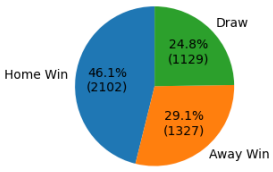
- Using Dropout layers between each hidden layers
- Using Early-Stop function
- Preventing the structure from becoming too complex (We limited the number of layers at 4)

The F1 score of our model on the training data set is 0.445 and the F1 score of our model on the testing data set is 0.446. Because of the similar performances on training data set and testing data set, we are confident that our model is not over-fitting.



A B

(a) DNN performance before handling over-fitting



(b) Distribution of results

Figure 5

5 Results

5.1 Model Selection

We realized that the training set was unbalanced, so (Figure 5b), selecting the model with the highest accuracy via cross validation might lead to a biased model that favors the result with the highest population (Home Win in our case). In order to evaluate the models more fairly, we analyzed and compared the F1 scores of each model.

Since the football result is a multi-class classification problem (result = {H,A,D}), the F1-scores of our models are calculated as the average of F1-scores for each class as follows:

$$F_1 = \frac{1}{3}(F_1(\text{class} = H) + F_1(\text{class} = A) + F_1(\text{class} = D))$$

The F1-score of each individual class is calculated in one-against-all fashion as follows:

$$Precision_S = \frac{\text{Number of samples correctly classified as S}}{\text{Number of S samples}}$$

$$Recall_S = \frac{\text{Number of samples correctly classified as S}}{\text{Number of samples classified as S}}$$

$$F_1(S) = \frac{2 * Precision_S * Recall_S}{Precision_S + Recall_S}$$

Moreover, we also analyzed the cross-validation accuracy and roc_auc score of each model to evaluate how well they generalize to future data. The accuracy, F1-score, roc_auc score and cross-validation accuracy of the models are listed (Table 5). The roc_auc score for the Stochastic Gradient Descent model is "N/A", is because our optimised loss function is a "hinge" function. The DNN only has scores for Accuracy and the F1 score because we could not obtain the other two scores from the Keras API we used. As we do not have a large feature-set, performing PCA did not improve the metric scores for most models, leading us to consider each model in a case by case basis. DNN outperforms all other models; it is at least 0.505 accurate, has the potential to achieve 0.55 accuracy and has a 0.51 F1 score - the best across the 9 models . We decided to select the top 3 models according to their F1 scores, which are DNN, Random Forest Classifier, and Linear Discriminant Analysis. We chose to use a combination of models, as they would act as committees [15], which would help dramatically reduce the error of our final prediction.

We noticed that even though we used same hyperparameters, the performance of DNN models varied after each training. This could be due to: 1) the train_test_split() function randomly shuffling the dataset, and 2) the initial weights of each layer were randomly initialized by using HeUniform() initializer.

6 Final Predictions

We noticed that one team in the test set, Leeds, did not have any previous match records in the training set. Because we used features such as Elo ratings and recency results, our approach depends heavily on historical performances of each teams, so, the lack of historical data could could have negative impact on the accuracy of our predictions.

Table 5: Model Scores Post-optimisation, Post-PCA & Comparisons

Model	Accuracy	F1 score	cv_score	roc_auc score	Mean Post-GridSearch	Post-PCA	Pre-Optimisation
LR	**0.527**	**0.469**	0.526	0.647	**0.542**	**0.539**	**0.542**
SGD	**0.525**	**0.466**	0.520	N/A	**0.502**	**0.547**	**0.389**
DTC	**0.408**	**0.410**	0.438	0.541	**0.449**	**0.441**	**0.448**
RFC	**0.542**	**0.472**	0.531	0.666	**0.553**	**0.547**	**0.542**
LDA	**0.534**	**0.481**	0.527	0.655	**0.549**	**0.540**	**0.549**
QDA	**0.495**	**0.480**	0.497	0.641	**0.528**	**0.532**	**0.496**
KNN	**0.461**	**0.403**	0.478	0.558	**0.475**	**0.492**	**0.453**
SVM	**0.535**	**0.449**	0.527	0.650	**0.540**	**0.537**	**0.538**
DNN	**0.505 – 0.550**	**0.410 – 0.510**	N/A	N/A	**0.496**	**0.454**	**0.473**

6.1 Make Final Predictions

In the "epl-test.csv" file, there were only three columns: Date, HomeTeam, AwayTeam, so we added in our new features before feeding them into our trained models. We then normalised the test data, using the same "StandardScaler" fitted by our training data. The predictions made by selected models are listed below.

Table 6: Final Predictions with the top 3 performing Models

Matches / Models	1	2	3	4	5	6	7	8	9	10
DNN	H	A	A	H	H	H	H	A	H	H
RFC	H	H	A	H	H	H	H	A	H	H
LDA	H	A	A	H	H	H	H	A	H	H
Final Predictions	H	A	A	H	H	H	H	A	H	H

From the table, we can see that the predictions made by selected models are almost same except for the second game. DNN and LDA prediced AwayTeam will win which RFC predicted HomeTeam will win, and we went with the majority. Hence:

Date	HomeTeam	AwayTeam	FTR
16-Jan-21	Arsenal	Newcastle	H
16-Jan-21	Aston Villa	Everton	A
16-Jan-21	Fulham	Chelsea	A
16-Jan-21	Leeds	Brighton	H
16-Jan-21	Leicester	Southampton	H
16-Jan-21	Liverpool	Man United	H
16-Jan-21	Man City	Crystal Palace	H
16-Jan-21	Sheffield United	Tottenham	A
16-Jan-21	West Ham	Burnley	H
16-Jan-21	Wolves	West Brom	H

Figure 6: Final Predictions

7 Conclusion

In this project, we tried to predict the results of the matches before 16th of January based on both the data provided to us, and the data we scraped from the Internet. We created features based on the data we were given using algorithms such as the Elo rating and the recency features. We also used some data preprocessing techniques such as one-hot encoding and normalization. In addition, we used NLP to analyse players' tweets before each match to generate new features that could potentially help improve the model's performance. We tested and tuned a variety of different models

by measuring their performance based on metric scores such as the F1-score and cross validation score. Furthermore, we tuned our model's hyperparameters using GridSearchCV to obtain the optimal hyperparameters.

For the DNN model we tried to fix problems such as over-fitting by attempting different activation functions and adding dropout layers. Ultimately we chose a combination of the models that had the best performance to act as a committee to predict the final results.

We believe that COVID-19 has fundamentally influenced some of the factors that affect the outcomes of football games. For example, not having fans in the stadium seems to have diminished the power of home teams to win [14]. Because of these changes, we tried to engineer features such as ELO ratings to compensate for varying performances of the football teams as a result of the differences in the environment. However, we believe an interesting area that could be further researched would be to look at models that place a larger significance on more recent data, which would be better suited to capture the different environments in which football games are carried out.

References

[1] Imbalanced Classification Problems, Jason Brownlee, 2019, available online at: https://bit.ly/35Or2Dam Accessed: 11 Jan, 2021

[2] Elo rating system, available online at: https://en.wikipedia.org/wiki/Elo_rating_system Accessed: 11 Jan, 2021

[3] How to Calculate the Elo-Rating (Including Examples) available online at: https://metinmediamath.wordpress.com/2013/11/27/how-to-calculate-the-elo-rating-including-example/ Accessed: 11 Jan, 2021

[4] Berrar, D., Lopes, P. and Dubitzky, W. (2018) Incorporating domain knowledge in machine learning for soccer outcome prediction, Machine Learning (2019) 108, pp.97–126, doi: 10.1007/s10994-018-5747-8

[5] Jerry Milani (2019), The Power of Mindset on Sports Performance, available online at: https://www.sportsmd.com/2019/04/04/the-power-of-mindset-on-sports-performance/ Accessed: 11 Jan, 2021

[6] Hutto, C.J. and Gilbert, E.E. (2014), VADER: A Parsimonious Rule-based Model for Sentiment Analysis of Social Media Text, Eighth International Conference on Weblogs and Social Media (ICWSM-14), Ann Arbor, MI, June 2014.

[7] L. B. Rosenberg, Human swarming, a real-time method for parallel distributed intelligence, 2015 Swarm/Human Blended Intelligence Workshop (SHBI), Cleveland, OH, 2015, pp. 1-7, doi: 10.1109/SHBI.2015.7321685.

[8] Brownlee, Jason (2014), Classification Accuracy is Not Enough: More Performance Measures You Can Use, available online at: https://machinelearningmastery.com/classification-accuracy-is-not-enough-more-performance-measures-you-can-use/, Accessed: 11 Jan, 2021

[9] Buraimo, Babatunde, et al. "The 12th Man?: Refer eeing Bias in English and German Soccer." Journal of the Royal Statistical Society. Series A (Statistics in Society), vol. 173, no. 2, 2010, pp. 431–449. JSTOR, www.jstor.org/stable/40666253. Accessed 16 Jan. 2021.

[10] Aishwarya V Srinivasan (2019), Why exclude highly correlated features when building regression model??, available online at: https://towardsdatascience.com/why-exclude-highly-correlated-features-when-building-regression-model-34d77a90ea8e, Accessed: 11 Jan, 2021

[11] What are the Pros and cons of the PCA? (2019), available online at: https://www.i2tutorials.com/what-are-the-pros-and-cons-of-the-pca/, Accessed: 11 Jan, 2021

[12] Jeff Heaton, Introduction to Neural Networks for Java (2008)

[13] Neural Networks Part 1: Setting up the Architecture, available online at: https://cs231n.github.io/neural-networks-1/, Accessed: 11 Jan, 2021

[14] EPL 2020/21 Season Analysis and Prediction, available online at: https://towardsdatascience.com/epl-2020-21-season-analysis-and-prediction-5502e20dce26, Accessed: 11 Jan, 2021

[15] Bishop, 2006, Pattern Recognition and Machine Learning